

Automation System Design

Conveyor Belt with Sensors: Counting and Metal / Non-Metal Sorting

1. Objective

This report presents the design of an automated conveyor system that moves objects along a belt, detects and counts them, identifies whether each object is metallic, and sorts metal objects into a separate bin. Automatic sorting like this is common in packaging, recycling and manufacturing lines, where it replaces slow and error-prone manual handling. The design is delivered as a control flowchart and a wiring diagram, both included in this report and supplied as separate design files.

2. System Description

Objects are placed at the start of a DC-motor-driven belt. An infrared sensor at the entry detects each object and triggers the counter. After a fixed travel time, the object reaches the diverter station, where an inductive proximity sensor checks for metal. If metal is detected, a servo-driven arm pushes the object into Bin B; otherwise the object continues to Bin A at the end of the belt. An Arduino Uno reads the sensors, controls the motor and servo, and shows the counts on an LCD. An emergency-stop button, a jam timeout, status LEDs and a buzzer provide basic safety and feedback.

Component	Role	Key detail
Arduino Uno	Main controller	ATmega328P, 5 V logic, runs the control program
IR obstacle sensor E18-D80NK	Detects objects at belt entry	NPN output, active LOW, 5 V supply
Inductive sensor LJ12A3-4-Z/BX	Detects metal objects	NPN output, active LOW, 6-36 V supply
L298N driver + DC gear motor	Drives the belt	PWM speed control on ENA, direction on IN1/IN2
Servo (SG90 / MG996R)	Diverter arm	0° = straight, 90° = push to Bin B
E-stop and reset buttons	Safety and recovery	E-stop uses a normally closed (NC) contact
LEDs, buzzer, I2C LCD 16x2	Status and counts	Green = running, red + buzzer = fault
12 V adapter + 5 V buck converter	Power	12 V for motor and inductive sensor; 5 V for servo and logic

Table 1. Main components of the system.

3. Design Constraints and Assumptions

The design assumes objects are placed one at a time with enough spacing for the diverter arm to return, and that belt speed is constant so a fixed travel time T_1 can be used between the entry sensor and the diverter station. T_1 and the diverter push time T_2 are calibrated on the real belt. A mixed-material, closely packed flow would need an additional sensor at the diverter or an encoder on the belt for position tracking.

4. Control Flowchart and Working

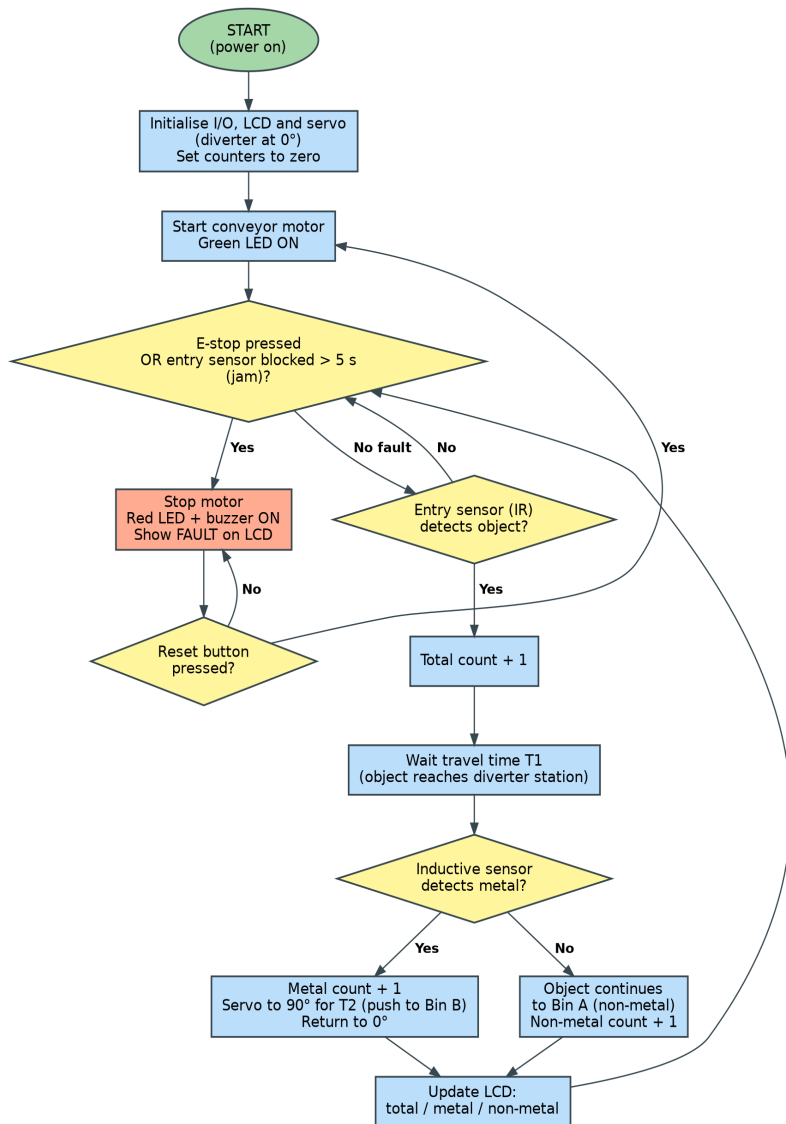


Figure 1. Control flowchart of the conveyor sorting system (design file: conveyor_flowchart.pdf / .svg / .dot).

1. **Start-up.** On power-up the controller sets up its inputs and outputs, moves the diverter to 0° and resets the three counters.
2. **Run.** The belt motor starts and the green LED lights.
3. **Safety check.** Every cycle the program checks the E-stop and a jam condition (entry sensor blocked for more than 5 s). If either is true, the motor stops, the red LED and buzzer turn on and the LCD shows FAULT. The system waits for the reset button before restarting.
4. **Detect.** If no fault exists, the program waits for the IR sensor to see an object, then adds one to the total count.
5. **Travel delay.** It waits for time T1 so the object arrives at the diverter station.
6. **Classify.** The inductive sensor is read. Metal: the metal count increases, the servo swings to 90° for time T2 to push the object into Bin B, then returns to 0°. Non-metal: the object travels on to Bin A and the non-metal count increases.
7. **Display.** The LCD shows total, metal and non-metal counts, and the loop returns to the safety check.

5. Wiring Diagram

Conveyor sorting system - wiring / block diagram

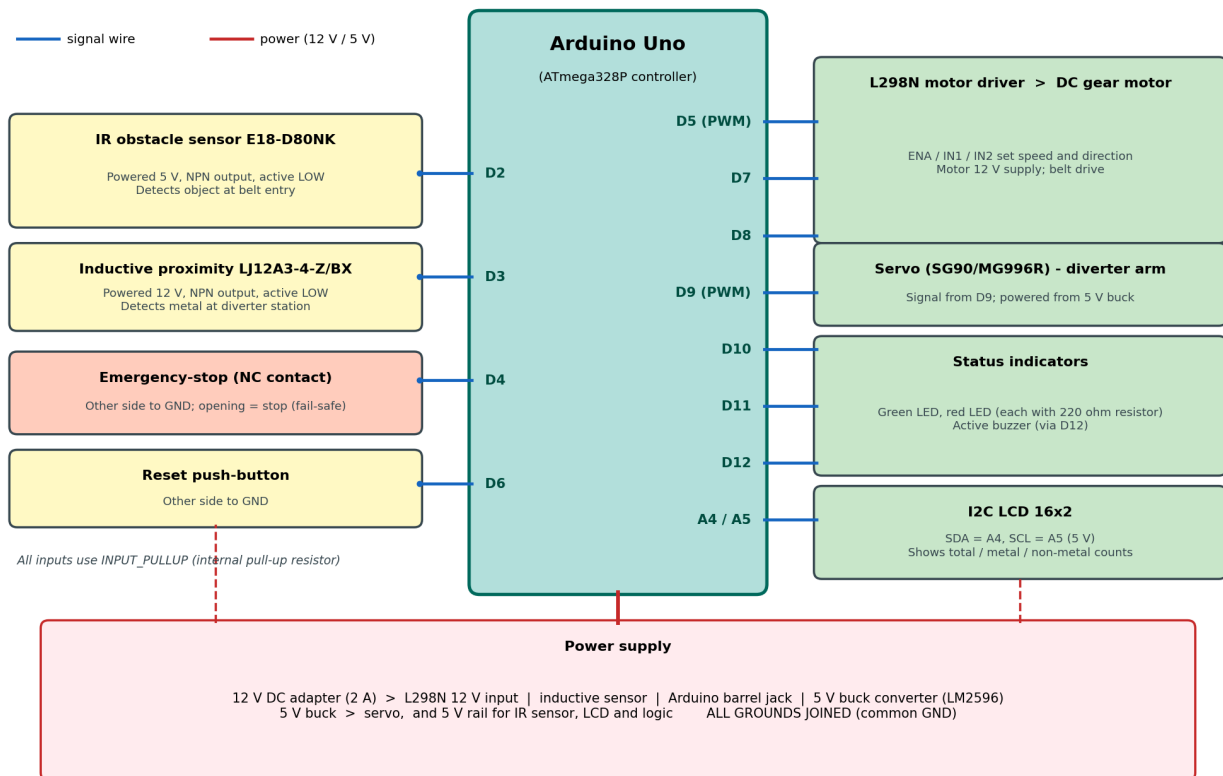


Figure 2. Wiring / block diagram with Arduino pin assignments (design file: wiring_diagram.svg / .png).

Both sensors have NPN open-collector outputs that pull the signal to ground when triggered, so they connect directly to Arduino inputs configured with `INPUT_PULLUP` and read LOW when active. The E-stop uses a normally closed contact: pressing it, or a broken wire, opens the circuit and the controller reads a stop condition (fail-safe). The servo is powered from the 5 V buck converter rather than the Arduino's 5 V pin to avoid voltage dips when the servo moves, and all grounds are joined.

6. Safety, Advantages and Limitations

Safety: fail-safe E-stop, jam timeout, audible and visual fault indication, and a manual reset requirement before restart. **Advantages:** low cost, simple logic, fast and consistent sorting, and an automatic count of items handled. **Limitations:** timing-based tracking depends on constant belt speed; only metal and non-metal are distinguished; a mechanical guard and a properly rated E-stop circuit would be needed for a real installation. Future improvements include a belt encoder, a camera for colour or size sorting, and a PLC for industrial use.

References

- [1] M. P. Groover, *Automation, Production Systems, and Computer-Integrated Manufacturing*, 4th ed. Pearson, 2015.
- [2] W. Bolton, *Programmable Logic Controllers*, 6th ed. Newnes (Elsevier), 2015.
- [3] Arduino, "Arduino Uno Rev3," Arduino Documentation. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/>